

Smart Contract Security Analysis Report

Generated on: 2026-04-24 02:00:16

1. Executive Summary

This report provides a consolidated view of the security analysis performed on 10 smart contract(s). In total, 8 potential vulnerabilities were identified across 8 contract(s).

2. Findings Overview

Contract Name	Total Issues	Status
TestSafeAccessControl.sol	0	SECURE
TestSafeMath.sol	1	VULNERABLE
TestSafePriceOracle.sol	1	VULNERABLE
TestSafeReentrancyGuard.sol	0	SECURE
TestSafeVault.sol	1	VULNERABLE
TestVulnAccessControl.sol	1	VULNERABLE
TestVulnFlashLoan.sol	1	VULNERABLE
TestVulnOverflow.sol	1	VULNERABLE
TestVulnPriceOracle.sol	1	VULNERABLE
TestVulnReentrancy.sol	1	VULNERABLE

3. Detailed Vulnerability Analyses

Contract: TestSafeMath.sol

Static Analysis Findings (Slither)

[MEDIUM] Potential Semantic / Business Logic Vulnerability (ML)

Function: Multiple Functions | Line: N/A (Semantic)

Explanation:

Smart Contract Security Analysis Report

Generated on: 2026-04-24 02:00:16

RandomForest ML model detected possible complex vulnerability with 90.1% confidence.

Suggest

Manually review business logic, reward calculations, price oracles, flash loan protections, and economic invariants.

Contract: TestSafePriceOracle.sol

Static Analysis Findings (Slither)

[MEDIUM] Timestamp Dependence

Function: getPrice | Line: 21

Explanation:

Function 'getPrice' uses 'block.timestamp' in conditional logic at line 21. Miners/validators can manipulate this value by up to ~15 seconds, potentially influencing time-sensitive operations such as timelocks, auction deadlines, or random-seed generation.

Suggest

Avoid relying on block.timestamp for exact timing or randomness. For timelocks, use sufficiently large windows (> 15 minutes) so miner manipulation is economically irrelevant. For randomness, use a commit-reveal scheme or a verifiable random function (VRF) such as Chainlink VRF.

Contract: TestSafeVault.sol

Static Analysis Findings (Slither)

[MEDIUM] Potential Semantic / Business Logic Vulnerability (ML)

Function: Multiple Functions | Line: N/A (Semantic)

Explanation:

RandomForest ML model detected possible complex vulnerability with 89.4% confidence.

Suggest

Manually review business logic, reward calculations, price oracles, flash loan protections, and economic invariants.

Contract: TestVulnAccessControl.sol

Static Analysis Findings (Slither)

[CRITICAL] Unprotected Self-Destruct

Function: destroy | Line: 14

Explanation:

Function 'destroy' contains a selfdestruct call at line 14 and has no access-control guard. Any external address can call this function, permanently destroying the contract and forwarding its entire ETH balance to an arbitrary recipient.

Suggest

Add an 'onlyOwner' modifier (or equivalent) to 'destroy', or replace the selfdestruct pattern with a pausable/upgradeable design. If self-destruction is truly required, protect it with: require(msg.sender == owner, "Not owner").

Smart Contract Security Analysis Report

Generated on: 2026-04-24 02:00:16

Contract: TestVulnFlashLoan.sol

Static Analysis Findings (Slither)

[MEDIUM] Potential Semantic / Business Logic Vulnerability (ML)

Function: Multiple Functions | Line: N/A (Semantic)

Explanation:

RandomForest ML model detected possible complex vulnerability with 95.0% confidence.

Manually review business logic, reward calculations, price oracles, flash loan protections, and economic invariants.

Suggest

Contract: TestVulnOverflow.sol

Static Analysis Findings (Slither)

[MEDIUM] Potential Semantic / Business Logic Vulnerability (ML)

Function: Multiple Functions | Line: N/A (Semantic)

Explanation:

RandomForest ML model detected possible complex vulnerability with 90.1% confidence.

Manually review business logic, reward calculations, price oracles, flash loan protections, and economic invariants.

Suggest

Contract: TestVulnPriceOracle.sol

Static Analysis Findings (Slither)

[MEDIUM] Potential Semantic / Business Logic Vulnerability (ML)

Function: Multiple Functions | Line: N/A (Semantic)

Explanation:

RandomForest ML model detected possible complex vulnerability with 78.0% confidence.

Manually review business logic, reward calculations, price oracles, flash loan protections, and economic invariants.

Suggest

Contract: TestVulnReentrancy.sol

Static Analysis Findings (Slither)

[HIGH] Reentrancy (CEI Violation)

Function: withdraw | Line: 16

Explanation:

Function 'withdraw' makes an external call before updating all state variables. A malicious contract can reenter this function

Smart Contract Security Analysis Report

Generated on: 2026-04-24 02:00:16

and exploit the stale state.

Follow the Checks-Effects-Interactions (CEI) pattern: move all state variable updates BEFORE the external call, or use a ReentrancyGuard modifier.

Suggest